

DIPLOMADO DE DEVOPS · MÓDULO 5

Proyecto Final

DevOps Monitoring en Azure Kubernetes Service

Infraestructura con Terraform · Prometheus, Grafana y Loki con Helm · Validación de un microservicio

Campo	Detalle
Autor	Rys Daybid Acuña Calderón
Repositorio público	https://github.com/Rac4na/devops-monitoring-aks
Clúster	aks-acuna-dev-eastus-01 (East US)
Stack	Terraform · AKS · Helm · Prometheus · Grafana · Loki

Contenido

1. Objetivo y alcance
2. Repositorio del proyecto
3. Infraestructura: clúster AKS con Terraform
4. Herramientas de observabilidad instaladas con Helm
5. Teoría: herramientas Open Source vs. de pago
6. Teoría: beneficios de DevOps Monitoring
7. Práctica: microservicio, logs y métricas en Grafana
8. Conclusiones
9. Anexos

1. Objetivo y alcance

El objetivo del proyecto es documentar y publicar un entorno de monitoreo sobre Azure Kubernetes Service (AKS), creado con Terraform, e instalar con Helm las herramientas del stack de observabilidad Prometheus, Grafana y Loki. Como parte de la sección práctica se despliega un microservicio y se valida su impacto en logs y métricas dentro de Grafana.

Todo el trabajo se realizó como código: la infraestructura en Terraform, la configuración de Grafana (data sources y dashboards) en valores de Helm y archivos JSON, y el microservicio en manifiestos de Kubernetes. De esta forma el entorno completo puede recrearse desde el repositorio con los comandos documentados.

1.1 Arquitectura de la solución

Capa	Componente	Rol en la solución
Infraestructura	Resource Group + AKS (Terraform, provider azurearm 4.56)	Clúster Kubernetes 1.35 con un node pool de 1 nodo Standard_D2als_v7
Métricas	Prometheus + node-exporter + kube-state-metrics	Recolecta métricas de nodos, pods, API server y del microservicio (/metrics)
Logs	Loki + Promtail (DaemonSet)	Promtail lee los logs de todos los contenedores del nodo y los envía a Loki
Visualización	Grafana (LoadBalancer, IP pública)	Data sources Prometheus y Loki aprovisionados; dashboards de nodos/pods y de logs
Carga de prueba	Microservicio podinfo + generador de tráfico	Deployment + Service LoadBalancer en el namespace dev; tráfico HTTP 200 y 404

1.2 Nomenclatura según el estándar de Azure

Los recursos siguen la convención del Cloud Adoption Framework de Azure: <tipo>-<carga de trabajo>-<entorno>-<región>-<instancia>. Las abreviaturas de tipo (rg, aks) son las recomendadas por Microsoft.

Recurso	Nombre	Descomposición
Resource Group	rg-acuna-dev-eastus-01	rg = resource group · acuna = carga/propietario · dev = desarrollo · eastus = región · 01 = instancia
Clúster AKS	aks-acuna-dev-eastus-01	aks = Azure Kubernetes Service, mismo sufijo que el grupo
Node resource group	MC_rg-acuna-dev-eastus-01_aks-acuna-dev-eastus-01_eastus	Generado automáticamente por AKS para la infraestructura de los nodos
Namespaces	monitoring · dev	Separan el stack de observabilidad del microservicio de prueba
Releases de Helm	prometheus · loki · grafana	Un release por herramienta dentro del namespace monitoring

2. Repositorio del proyecto

El código Terraform, los valores de Helm, los manifiestos de Kubernetes, los dashboards y las evidencias están publicados en el repositorio público: <https://github.com/Rac4na/devops-monitoring-aks>

2.1 Estructura del repositorio

Ruta	Contenido
provider.tf, aks.tf	Terraform: provider azurearm, resource group y clúster AKS
helm/grafana-values.yaml	Valores de Helm para Grafana: servicio LoadBalancer, data sources Prometheus y Loki, aprovisionamiento de dashboards
dashboards/aks-dashboard.json	Dashboard "AKS - Monitoreo de Nodos y Pods" (CPU, memoria, disco, red, pods por estado)
dashboards/logs-dashboard.json	Dashboard "Microservicio - Logs y Requests" (Loki + Prometheus)
k8s/microservicio.yaml	Namespace dev, Deployment podinfo (3 réplicas) y Service LoadBalancer
k8s/load-generator.yaml	Deployment que genera tráfico HTTP continuo hacia el microservicio
evidencias/	Salidas de comandos, capturas de Grafana y gráficos antes/después
comandos.md, README.md	Guía paso a paso de todos los comandos ejecutados
docs/	Este documento y el enunciado del proyecto

3. Infraestructura: clúster AKS con Terraform

El clúster se aprovisiona con dos recursos de Terraform: el resource group y el clúster AKS con identidad administrada por el sistema (SystemAssigned), un node pool por defecto de un nodo y etiquetas de entorno. El tamaño de VM elegido es Standard_D2as_v7 (2 vCPU, 4 GiB), suficiente para el stack de monitoreo y el microservicio de prueba con un costo bajo.

provider.tf

```

terraform {
  required_providers {
    azurearm = {
      source = "hashicorp/azurearm"
      version = "=4.56.0"
    }
  }
}

# Configure the Microsoft Azure Provider
provider "azurearm" {
  resource_provider_registrations = "none"
  features {}
  subscription_id = "e3397c4a-0aab-4d1a-8d2d-68380faf6e56"
}

```

aks.tf

```

resource "azurerm_resource_group" "rg_01" {
  name      = "rg-acuna-dev-eastus-01"
  location  = "East US"
  tags = {
    Project = "Monitoring"
  }
}

resource "azurerm_kubernetes_cluster" "aks_01" {
  name                = "aks-acuna-dev-eastus-01"
  location            = azurerm_resource_group.rg_01.location
  resource_group_name = azurerm_resource_group.rg_01.name
  dns_prefix          = "aksdns"

  default_node_pool {
    name       = "default"
    node_count = 1
    vm_size    = "Standard_D2as_v7"

    upgrade_settings {
      max_surge = "10%"
    }
  }

  oidc_issuer_enabled = true

  identity {
    type = "SystemAssigned"
  }

  tags = {
    Environment = "Development"
  }
}

output "client_certificate" {
  value     = azurerm_kubernetes_cluster.aks_01.kube_config[0].client_certificate
  sensitive = true
}

output "kube_config" {
  value = azurerm_kubernetes_cluster.aks_01.kube_config_raw

  sensitive = true
}

```

3.1 Comandos ejecutados

```

terraform init
terraform plan
terraform apply
az aks get-credentials --resource-group rg-acuna-dev-eastus-01 --name aks-acuna-dev-eastus-01 --
overwrite-existing
kubectl get nodes -o wide

```

3.2 Evidencia

Tras el apply, un nuevo terraform plan confirma que la infraestructura real coincide con el código (sin drift). Azure CLI y kubectl muestran el clúster en estado Succeeded/Running y el nodo Ready.

evidencias/01-terraform-plan.txt

```
$ terraform plan
No changes. Your infrastructure matches the configuration.
Terraform has compared your real infrastructure against your configuration
and found no differences, so no changes are needed.

$ terraform state list
azurerm_kubernetes_cluster.aks_01
azurerm_resource_group.rg_01
```

evidencias/02-az-aks.txt

```
$ az aks list -o table
Name                               ResourceGroup      Location    State    Power    K8s    VmSize
Nodes
-----
aks-acuna-dev-eastus-01 rg-acuna-dev-eastus-01 eastus      Succeeded Running 1.35
Standard_D2as_v7 1

$ az group list -o table
Name                               Location    State
-----
rg-acuna-dev-eastus-01 eastus      Succeeded
MC_rg-acuna-dev-eastus-01_aks-acuna-dev-eastus-01_eastus eastus      Succeeded
```

evidencias/03-kubectl-nodes.txt

```
$ kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE    VERSION    INTERNAL-IP    EXTERNAL-IP
OS-IMAGE      KERNEL-VERSION    CONTAINER-RUNTIME
aks-default-12912847-vmss000000 Ready    <none>    53m    v1.35.8    10.224.0.4    <none>
Ubuntu 24.04.5 LTS 6.8.0-1067-azure containerd://2.3.3-2
```

4. Herramientas de observabilidad instaladas con Helm

Las tres herramientas se instalan desde sus charts oficiales en el namespace monitoring. Prometheus se despliega con node-exporter y kube-state-metrics (Alertmanager y Pushgateway se desactivan porque no se usan en el proyecto). Loki se instala con el chart loki-stack, que incluye Promtail como DaemonSet. Grafana se instala con un archivo de valores que deja el servicio expuesto por LoadBalancer y aprovisiona automáticamente los data sources y dashboards.

Release	Chart (versión)	Versión de la aplicación	Estado
grafana	grafana-10.5.15	12.3.1	deployed
loki	loki-stack-2.10.3	v2.9.3	deployed
prometheus	prometheus-29.33.0	v3.14.0	deployed

4.1 Prometheus

```
helm repo add prometheus-community https://prometheus-community.github.io/helm-charts
helm repo update
kubectl create namespace monitoring
helm upgrade --install prometheus prometheus-community/prometheus --namespace monitoring \
  --set alertmanager.enabled=false \
  --set prometheus-pushgateway.enabled=false \
  --set server.retention=3d
```

Prometheus descubre automáticamente los targets de Kubernetes (nodos, cAdvisor, API server, endpoints de servicios y pods anotados con prometheus.io/scrape). Queda accesible dentro del clúster en `http://prometheus-server.monitoring.svc.cluster.local:80`.

4.2 Loki y Promtail

```
helm repo add grafana https://grafana.github.io/helm-charts
helm repo update
helm upgrade --install loki grafana/loki-stack --namespace monitoring \
--set grafana.enabled=false --set promtail.enabled=true --set loki.image.tag=2.9.8
```

Promtail se ejecuta en cada nodo, etiqueta cada línea de log con namespace, pod, container y app, y la envía a Loki en `http://loki.monitoring.svc.cluster.local:3100`.

4.3 Grafana con data sources y dashboards como código

```
kubectrl create configmap grafana-dashboards -n monitoring \
--from-file=dashboards/aks-dashboard.json --from-file=dashboards/logs-dashboard.json
helm upgrade --install grafana grafana/grafana --namespace monitoring -f helm/grafana-values.yaml

# IP pública y contraseña de admin
kubectrl get svc grafana -n monitoring
kubectrl get secret grafana -n monitoring -o jsonpath="{.data.admin-password}" | base64 --decode
```

helm/grafana-values.yaml

```
# Valores de Helm para Grafana (chart grafana/grafana)
# Uso: helm upgrade --install grafana grafana/grafana -n monitoring -f helm/grafana-values.yaml
service:
  type: LoadBalancer

# Data sources aprovisionados automaticamente (Prometheus + Loki)
datasources:
  datasources.yaml:
    apiVersion: 1
    datasources:
      - name: Prometheus
        uid: prometheus
        type: prometheus
        access: proxy
        url: http://prometheus-server.monitoring.svc.cluster.local:80
        isDefault: true
      - name: Loki
        uid: loki
        type: loki
        access: proxy
        url: http://loki.monitoring.svc.cluster.local:3100

# Dashboards aprovisionados desde el ConfigMap "grafana-dashboards"
# (creado con: kubectrl create configmap grafana-dashboards -n monitoring --from-file=dashboards/)
dashboardProviders:
  dashboardproviders.yaml:
    apiVersion: 1
    providers:
      - name: default
        orgId: 1
        folder: ""
        type: file
        disableDeletion: false
        editable: true
        options:
          path: /var/lib/grafana/dashboards/default
dashboardsConfigMaps:
  default: grafana-dashboards
```

Grafana quedó publicada en <http://20.237.52.159> (IP pública asignada por el Azure Load Balancer durante la ejecución; cambia si se recrea el servicio). El usuario es admin y la contraseña se obtiene del secret generado por el chart.

4.4 Evidencia

evidencias/04-helm-list.txt

```
$ helm list -n monitoring
NAME                NAMESPACE    REVISION    UPDATED                               STATUS    CHART
APP VERSION
grafana             monitoring    1           2026-09-23 22:14:57.028979 -0500 -05    deployed
grafana-10.5.15     12.3.1
loki                monitoring    1           2026-09-23 22:13:08.731944 -0500 -05    deployed    loki-
stack-2.10.3        v2.9.3
prometheus          monitoring    1           2026-09-23 22:13:01.273105 -0500 -05    deployed
prometheus-29.33.0  v3.14.0

$ helm repo list
NAME                URL
prometheus-community  https://prometheus-community.github.io/helm-charts
grafana              https://grafana.github.io/helm-charts
```

evidencias/05-monitoring-pods-svc.txt

```
$ kubectl get pods -n monitoring
NAME                                READY   STATUS    RESTARTS   AGE
grafana-6d9b59c579-vgg9q            1/1     Running   0           62s
loki-0                               1/1     Running   0           5m47s
loki-promtail-2lx5l                 1/1     Running   0           5m47s
prometheus-kube-state-metrics-798fb7d7b8-dj729  1/1     Running   0           5m54s
prometheus-prometheus-node-exporter-nz4qd      1/1     Running   0           5m54s
prometheus-server-fcf964c77-8jl7w            2/2     Running   0           5m54s

$ kubectl get svc -n monitoring
NAME                                TYPE                CLUSTER-IP      EXTERNAL-IP      PORT(S)
AGE
grafana                             LoadBalancer       10.0.113.217    20.237.52.159    80:32569/TCP
4m
loki                                ClusterIP           10.0.49.141     <none>            3100/TCP
5m49s
loki-headless                       ClusterIP           None            <none>            3100/TCP
5m49s
loki-memberlist                     ClusterIP           None            <none>            7946/TCP
5m49s
prometheus-kube-state-metrics        ClusterIP           10.0.196.192    <none>            8080/TCP
5m56s
prometheus-prometheus-node-exporter ClusterIP           10.0.79.218     <none>            9100/TCP
5m56s
prometheus-server                    ClusterIP           10.0.235.234    <none>            80/TCP
5m56s
```

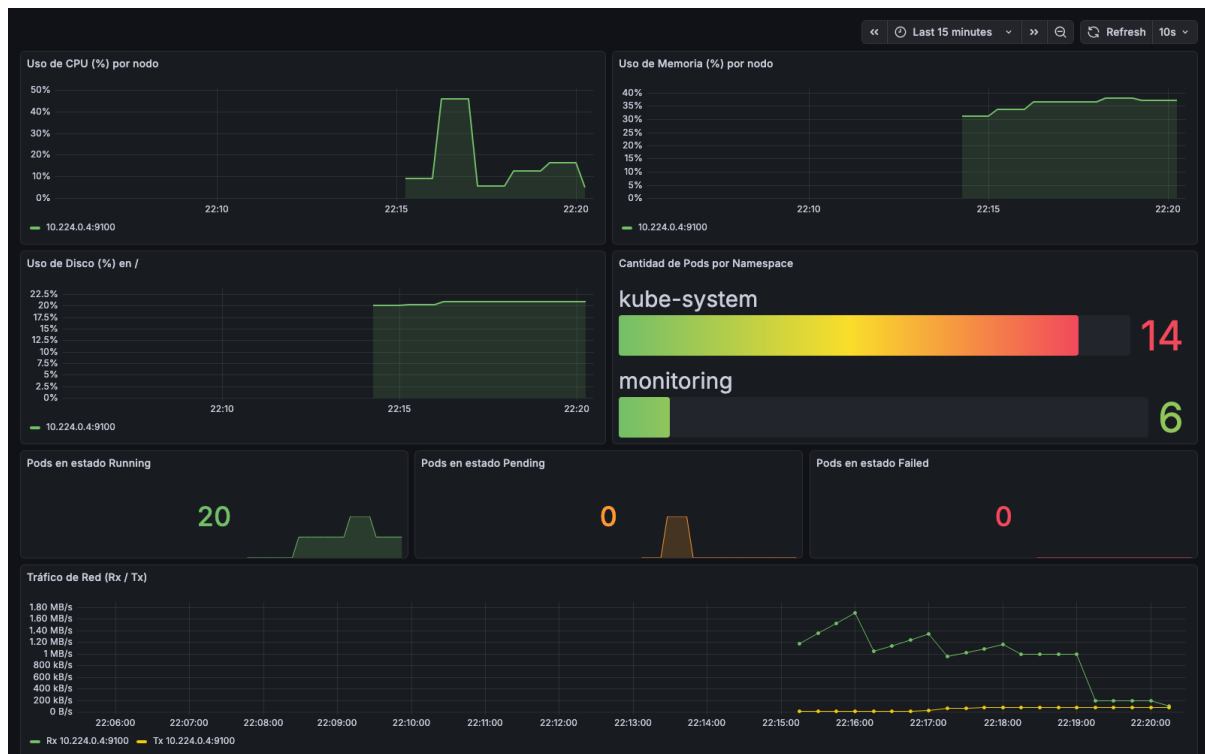


Figura 1. Dashboard "AKS - Monitoreo de Nodos y Pods" en Grafana con datos de Prometheus, antes de desplegar el microservicio.

5. Teoría: herramientas Open Source vs. de pago

En monitoreo conviven dos grandes familias de herramientas. Las Open Source (Prometheus, Grafana, Loki, Alertmanager, Zabbix, la pila ELK/OpenSearch, Jaeger, OpenTelemetry) se distribuyen con licencias libres y se operan por el propio equipo. Las de pago o comerciales (Datadog, New Relic, Dynatrace, Splunk, Elastic Cloud, Azure Monitor / Application Insights, Grafana Cloud) se consumen normalmente como servicio gestionado (SaaS) y se facturan por host, por GB ingerido, por usuario o por consultas.

Criterio	Open Source (Prometheus, Grafana, Loki)	De pago / SaaS (Datadog, New Relic, Dynatrace, Azure Monitor)
Costo de licencia	Cero. El costo real es la infraestructura y el tiempo del equipo.	Suscripción que crece con hosts, GB de logs y usuarios; puede volverse el mayor costo de plataforma.
Operación y mantenimiento	Lo asume el equipo: instalación, actualizaciones, almacenamiento, alta disponibilidad, retención.	Lo asume el proveedor; el equipo solo instala agentes y configura.
Tiempo de puesta en marcha	Horas o días; requiere conocimiento de Kubernetes, PromQL/LogQL y Helm.	Minutos; dashboards, alertas e integraciones vienen listos.
Flexibilidad y personalización	Total: código abierto, exporters para casi cualquier sistema, dashboards versionables como JSON.	Alta pero dentro de lo que ofrece el producto; extensiones limitadas al ecosistema del proveedor.
Funcionalidades avanzadas	Se construyen combinando piezas (trazas con Tempo/Jaeger, alertas con Alertmanager, SLOs con Sloth).	Integradas: APM, RUM, detección de anomalías con IA, mapas de servicios, seguridad, correlación automática.
Soporte	Comunidad, documentación y, opcionalmente, soporte comercial (Grafana Labs, Red Hat).	SLA contractual, soporte 24x7, gerentes de cuenta.

Criterio	Open Source (Prometheus, Grafana, Loki)	De pago / SaaS (Datadog, New Relic, Dynatrace, Azure Monitor)
Escalabilidad	Requiere diseño (federación, Thanos/Mimir, Loki distribuido).	Transparente para el cliente; se paga por volumen.
Seguridad y cumplimiento	Los datos permanecen en la infraestructura propia (útil para regulaciones de residencia de datos).	Los datos salen a la nube del proveedor; certificaciones (SOC 2, ISO 27001) incluidas.
Dependencia del proveedor	Baja: formatos abiertos (OpenMetrics, OpenTelemetry) y portabilidad.	Alta: agentes, dashboards y alertas propietarios; migrar es costoso.
Curva de aprendizaje	Media-alta (PromQL, LogQL, configuración YAML).	Baja para empezar; el conocimiento es específico del producto.

5.1 Cuándo conviene cada opción

- **Open Source:** equipos con conocimiento de Kubernetes que quieren control total y bajo costo de licencias, entornos de laboratorio y desarrollo, organizaciones con restricciones de residencia de datos, o plataformas que buscan evitar la dependencia de un proveedor. Es la opción natural en Kubernetes, donde Prometheus es el estándar de facto.
- **De pago:** equipos pequeños que no pueden dedicar personas a operar la plataforma de monitoreo, empresas que necesitan APM, trazas distribuidas y análisis avanzado desde el primer día, o entornos regulados que exigen SLA de soporte.
- **Híbrido:** cada vez más común. Se recolecta con estándares abiertos (Prometheus, OpenTelemetry) y se almacena/visualiza en un servicio gestionado (Grafana Cloud, Azure Managed Prometheus y Azure Managed Grafana). Se conserva la portabilidad y se delega la operación.

6. Teoría: beneficios de DevOps Monitoring

DevOps Monitoring es la práctica de instrumentar aplicaciones e infraestructura para obtener métricas, logs y trazas de forma continua, y usar esa información en todo el ciclo de vida del software: desde el pipeline de CI/CD hasta la operación en producción. Sus principales beneficios son:

- **Detección temprana y menor tiempo de recuperación (MTTR).** Las alertas basadas en métricas (CPU, memoria, errores 5xx, latencia) permiten detectar un problema antes de que lo reporte el usuario, y los logs centralizados aceleran el diagnóstico. En el proyecto, el impacto del microservicio se observó en Grafana en menos de un minuto.
- **Visibilidad de extremo a extremo (observabilidad).** Al correlacionar métricas (Prometheus), logs (Loki) y, opcionalmente, trazas, el equipo entiende no solo que algo falla, sino por qué. Un mismo panel muestra el consumo del nodo, los pods afectados y las líneas de log del error.
- **Ciclo de retroalimentación para CI/CD.** Cada despliegue se valida con datos: comparar CPU, memoria, tasa de errores y latencia antes y después de la versión permite hacer despliegues canary o blue/green y revertir automáticamente si las métricas empeoran.

- **Planificación de capacidad y control de costos (FinOps).** Conocer el uso real de recursos evita sobredimensionar nodos y permite ajustar requests/limits, autoescalado (HPA, Cluster Autoscaler) y tamaños de VM con evidencia, en lugar de estimaciones.
- **Cumplimiento de SLO y SLA.** Los indicadores de nivel de servicio (SLI) como disponibilidad, latencia p95 o tasa de errores se calculan a partir de las métricas recolectadas, y los presupuestos de error guían la prioridad entre nuevas funcionalidades y estabilidad.
- **Automatización y auto-remediación.** Las métricas alimentan el autoescalado, las alertas disparan runbooks o pipelines y los health checks de Kubernetes reinician contenedores degradados. El monitoreo deja de ser solo un panel para convertirse en un insumo de acciones automáticas.
- **Cultura de responsabilidad compartida.** Con dashboards accesibles para desarrollo y operaciones, ambos equipos comparten los mismos datos ("you build it, you run it"), los post-mortems se basan en evidencia y se reduce la fricción entre áreas.
- **Seguridad y auditoría.** Los logs centralizados y las métricas de acceso permiten detectar comportamientos anómalos (picos de 404/401, tráfico inusual) y conservar evidencia para auditorías.
- **Mejora continua basada en datos.** Las tendencias históricas revelan degradaciones lentas, cuellos de botella y oportunidades de optimización que no se ven en una prueba puntual.

7. Práctica: microservicio, logs y métricas en Grafana

Para validar el stack se desplegó el microservicio podinfo (escrito en Go) en el namespace dev con un Deployment de 3 réplicas y un Service de tipo LoadBalancer. podinfo expone una API REST en el puerto 9898, publica métricas Prometheus en /metrics y escribe logs estructurados en JSON a la salida estándar, por lo que es ideal para comprobar la cadena completa: Promtail recoge los logs y los envía a Loki, Prometheus recolecta las métricas del pod (anotación prometheus.io/scrape) y Grafana muestra ambos. Para simular usuarios se añadió un Deployment generador de tráfico que hace peticiones continuas a las rutas / (200), /api/info (200) y /status/404 (404).

7.1 Despliegue (Deployment + Service)

k8s/microservicio.yaml

```
# Microservicio de demostracion: podinfo (Go). Expone API REST en :9898,
# metricas Prometheus en /metrics y logs estructurados en stdout.
apiVersion: v1
kind: Namespace
metadata:
  name: dev
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: podinfo
  namespace: dev
  labels:
    app: podinfo
spec:
  replicas: 3
  selector:
    matchLabels:
      app: podinfo
  template:
    metadata:
      labels:
        app: podinfo
      annotations:
        prometheus.io/scrape: "true"
        prometheus.io/port: "9898"
        prometheus.io/path: "/metrics"
    spec:
      containers:
        - name: podinfo
          image: ghcr.io/stefanprodan/podinfo:6.7.1
          command: ["/podinfo"]
          args:
            - --port=9898
            - --level=debug
            - --stress-memory=64 # simula consumo de memoria (MB) para evidenciar el impacto
          ports:
            - name: http
              containerPort: 9898
          resources:
            requests:
              cpu: 50m
              memory: 96Mi
            limits:
              memory: 192Mi
          readinessProbe:
            httpGet:
              path: /readyz
              port: 9898
            initialDelaySeconds: 3
          livenessProbe:
            httpGet:
              path: /healthz
              port: 9898
            initialDelaySeconds: 5
---
apiVersion: v1
kind: Service
metadata:
  name: podinfo
  namespace: dev
  labels:
    app: podinfo
spec:
  type: LoadBalancer
  selector:
    app: podinfo
  ports:
    - name: http
      port: 80
      targetPort: 9898
```

```
kubectl apply -f k8s/microservicio.yaml
kubectl rollout status deployment/podinfo -n dev
kubectl apply -f k8s/load-generator.yaml
kubectl get deploy,pods,svc -n dev -o wide
```

Evidencia: pods en Running y servicio expuesto

evidencias/08-microservicio-running.txt

```
# Microservicio desplegado (2026-09-23 22:26:03 -05)
$ kubectl apply -f k8s/microservicio.yaml
namespace/dev created
deployment.apps/podinfo created
service/podinfo created

$ kubectl get deploy,pods,svc -n dev -o wide
NAME                                READY    UP-TO-DATE    AVAILABLE    AGE    CONTAINERS    IMAGES
SELECTOR
deployment.apps/load-generator      2/2      2              2            4s     load          app=load-generator
busybox:1.36
deployment.apps/podinfo            3/3      3              3            5m25s   podinfo       ghcr.io/stefanprodan/podinfo:6.7.1
ghcr.io/stefanprodan/podinfo:6.7.1  app=podinfo

NAME                                READY    STATUS    RESTARTS    AGE    IP            NODE
NOMINATED NODE    READINESS GATES
pod/load-generator-787b5586d6-hwtb6 1/1      Running   0           5s     10.244.0.101  aks-
default-12912847-vmss0000000    <none>   <none>     0           5s     10.244.0.60   aks-
pod/load-generator-787b5586d6-x6p4h 1/1      Running   0           5s     10.244.0.60   aks-
default-12912847-vmss0000000    <none>   <none>     0           31s    10.244.0.178  aks-
pod/podinfo-65f46f89-hwswc        1/1      Running   0           19s    10.244.0.95   aks-
default-12912847-vmss0000000    <none>   <none>     0           43s    10.244.0.250  aks-
pod/podinfo-65f46f89-lkvqr        1/1      Running   0           43s    10.244.0.250  aks-
default-12912847-vmss0000000    <none>   <none>     0           43s    10.244.0.250  aks-
pod/podinfo-65f46f89-mnbsq        1/1      Running   0           43s    10.244.0.250  aks-
default-12912847-vmss0000000    <none>   <none>     0           43s    10.244.0.250  aks-

NAME                                TYPE    CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE    SELECTOR
service/podinfo                    LoadBalancer  10.0.63.233    52.224.138.212  80:31065/TCP  5m26s  app=podinfo

$ curl -s http://52.224.138.212/
{
  "hostname": "podinfo-65f46f89-lkvqr",
  "version": "6.7.1",
  "revision": "6b7aab8a10d6ee8b895b0a5048f4ab0966ed29ff",
  "color": "#34577c",
  "logo": "https://raw.githubusercontent.com/stefanprodan/podinfo/gh-pages/cuddle_clap.gif",
  "message": "greetings from podinfo v6.7.1",
  "goos": "linux",
  "goarch": "amd64",
  "runtime": "go1.23.2",
  "num_goroutine": "6",
  "num_cpu": "2"
}

$ curl -s -o /dev/null -w '%{http_code}' http://52.224.138.212/status/404
404
```

El microservicio respondió en su IP pública (<http://52.224.138.212>) con HTTP 200 en la raíz y HTTP 404 en la ruta de prueba, y los 3 pods quedaron en estado Running.

7.2 Logs del microservicio en Loki / Grafana

Los logs se consultan con LogQL desde Explore o desde el dashboard "Microservicio - Logs y Requests". Las etiquetas namespace y app las añade Promtail automáticamente a partir de los metadatos del pod.

Consultas LogQL utilizadas

```
{namespace="dev", app="podinfo"} # todas las líneas del microservicio
{namespace="dev", app="podinfo"} |= "/status/404" # solo las peticiones 404
sum(count_over_time({namespace="dev", app="podinfo"}[1m])) # líneas de log por minuto
```

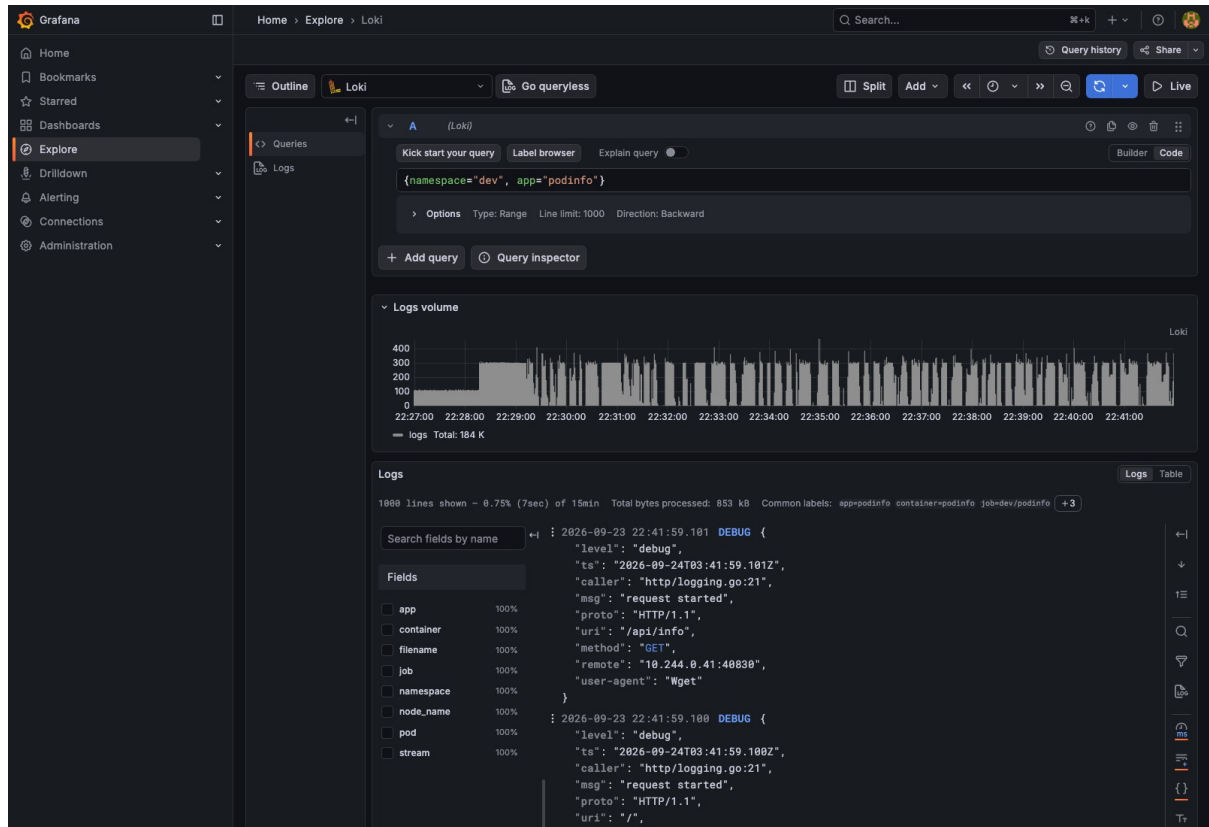


Figura 2. Grafana Explore con el data source Loki mostrando los logs JSON del microservicio podinfo (namespace dev).



Figura 3. Dashboard de logs: líneas por minuto, peticiones por código HTTP, peticiones 404 detectadas en logs y panel de logs en vivo.

evidencias/09-logs-kubectl-podinfo.txt (las mismas líneas que se ven en Loki)

```
$ kubectl logs -n dev deploy/podinfo --tail=8
{"level":"debug","ts":"2026-09-24T03:42:16.789Z","caller":"http/logging.go:21","msg":"request started","proto":"HTTP/1.1","uri":"/status/404","method":"GET","remote":"10.244.0.24:38608","user-agent":"Wget"}
{"level":"debug","ts":"2026-09-24T03:42:16.800Z","caller":"http/logging.go:21","msg":"request started","proto":"HTTP/1.1","uri":"/","method":"GET","remote":"10.244.0.24:38612","user-agent":"Wget"}
{"level":"debug","ts":"2026-09-24T03:42:16.802Z","caller":"http/logging.go:21","msg":"request started","proto":"HTTP/1.1","uri":"/","method":"GET","remote":"10.244.0.41:54092","user-agent":"Wget"}
{"level":"debug","ts":"2026-09-24T03:42:16.803Z","caller":"http/logging.go:21","msg":"request started","proto":"HTTP/1.1","uri":"/status/404","method":"GET","remote":"10.244.0.24:38636","user-agent":"Wget"}
{"level":"debug","ts":"2026-09-24T03:42:16.819Z","caller":"http/logging.go:21","msg":"request started","proto":"HTTP/1.1","uri":"/api/info","method":"GET","remote":"10.244.0.24:38654","user-agent":"Wget"}
{"level":"debug","ts":"2026-09-24T03:42:16.889Z","caller":"http/logging.go:21","msg":"request started","proto":"HTTP/1.1","uri":"/","method":"GET","remote":"10.244.0.24:38780","user-agent":"Wget"}
{"level":"debug","ts":"2026-09-24T03:42:16.890Z","caller":"http/logging.go:21","msg":"request started","proto":"HTTP/1.1","uri":"/api/info","method":"GET","remote":"10.244.0.24:38796","user-agent":"Wget"}
{"level":"debug","ts":"2026-09-24T03:42:16.902Z","caller":"http/logging.go:21","msg":"request started","proto":"HTTP/1.1","uri":"/","method":"GET","remote":"10.244.0.24:38816","user-agent":"Wget"}

$ kubectl logs -n dev deploy/podinfo --tail=300 | grep -c '/status/404'
99
```

7.3 Métricas: CPU y memoria antes y después del microservicio

El despliegue del microservicio se ejecutó a las 22:25. Se comparó el consumo del nodo en los minutos previos (solo el stack de monitoreo) con el consumo posterior (microservicio con 3 réplicas más el generador de tráfico). La comparación se hizo con el dashboard de Grafana y con consultas directas a Prometheus.

Métrica (fuente Prometheus)	Antes del despliegue	Después del despliegue	Variación
CPU del nodo (%)	12,8 %	37,8 %	+25,0 puntos
Memoria del nodo (%)	37,1 %	51,6 %	+14,5 puntos
CPU de los pods del namespace dev	0 m (no existía)	317 millicores	+317 m
Memoria de los pods del namespace dev	0 MiB (no existía)	434 MiB	+434 MiB

Promedios calculados con `query_range` (paso 15 s) sobre la ventana 22:13–22:42; "antes" excluye los 2 primeros minutos de la ventana y "después" los 2 minutos siguientes al apply.

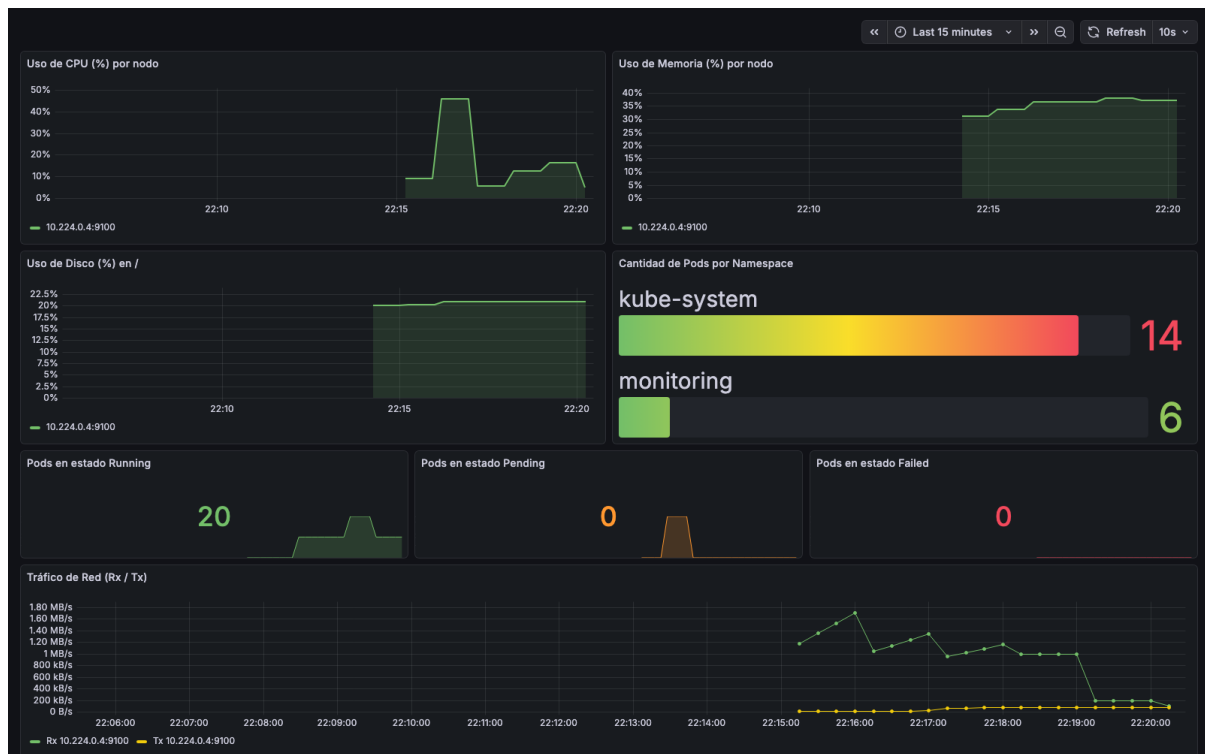


Figura 4. ANTES: dashboard de nodos en Grafana en los 15 minutos previos al despliegue. El pico inicial corresponde a la instalación del propio stack de monitoreo.

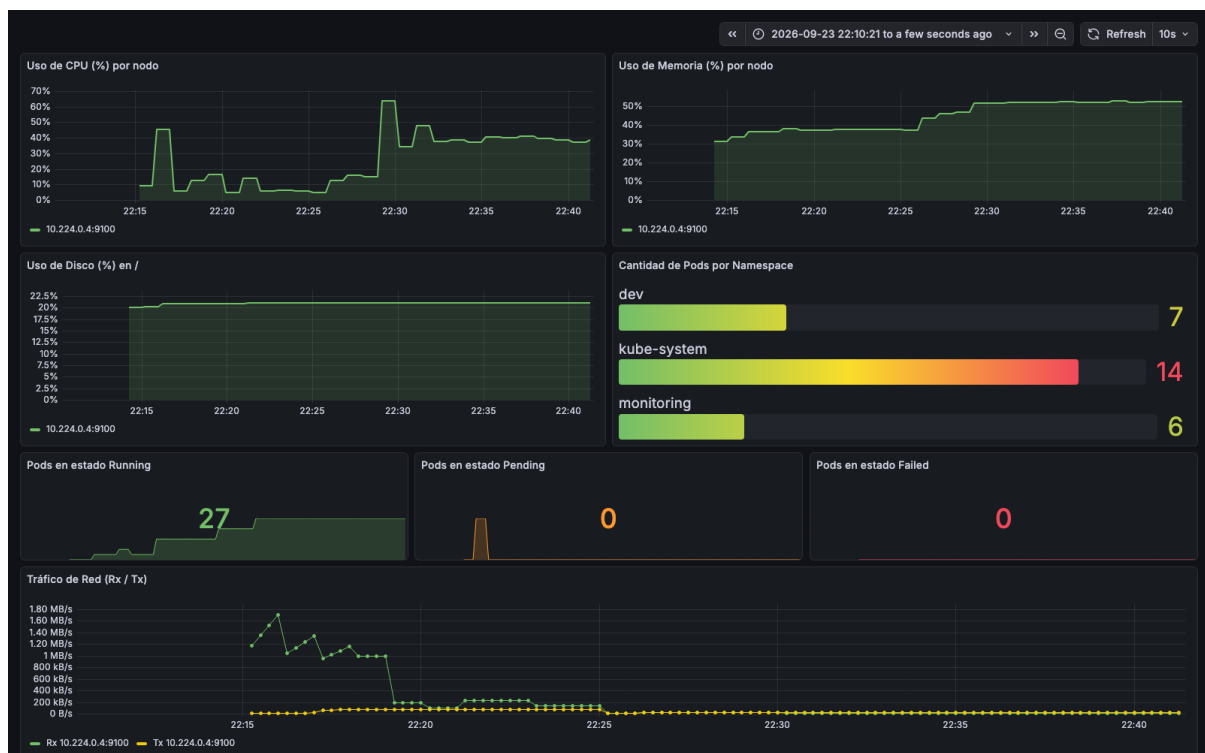
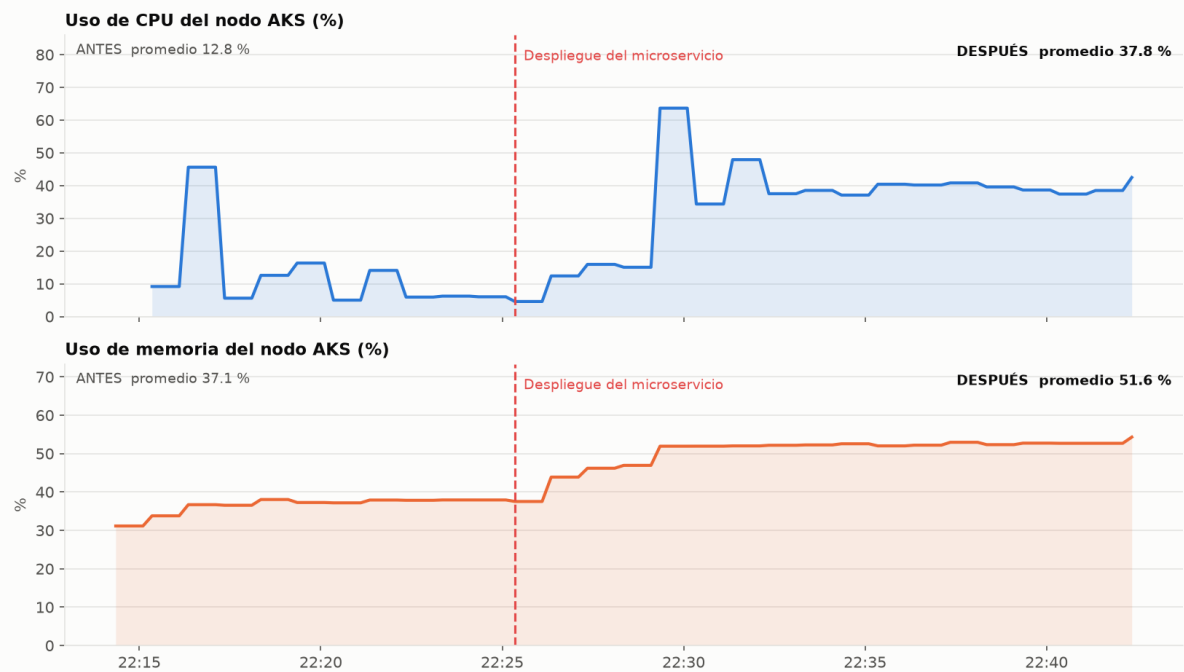


Figura 5. DESPUÉS: el mismo dashboard tras desplegar el microservicio y el generador de tráfico. Se aprecia el escalón de CPU y memoria y el aumento de pods en Running.

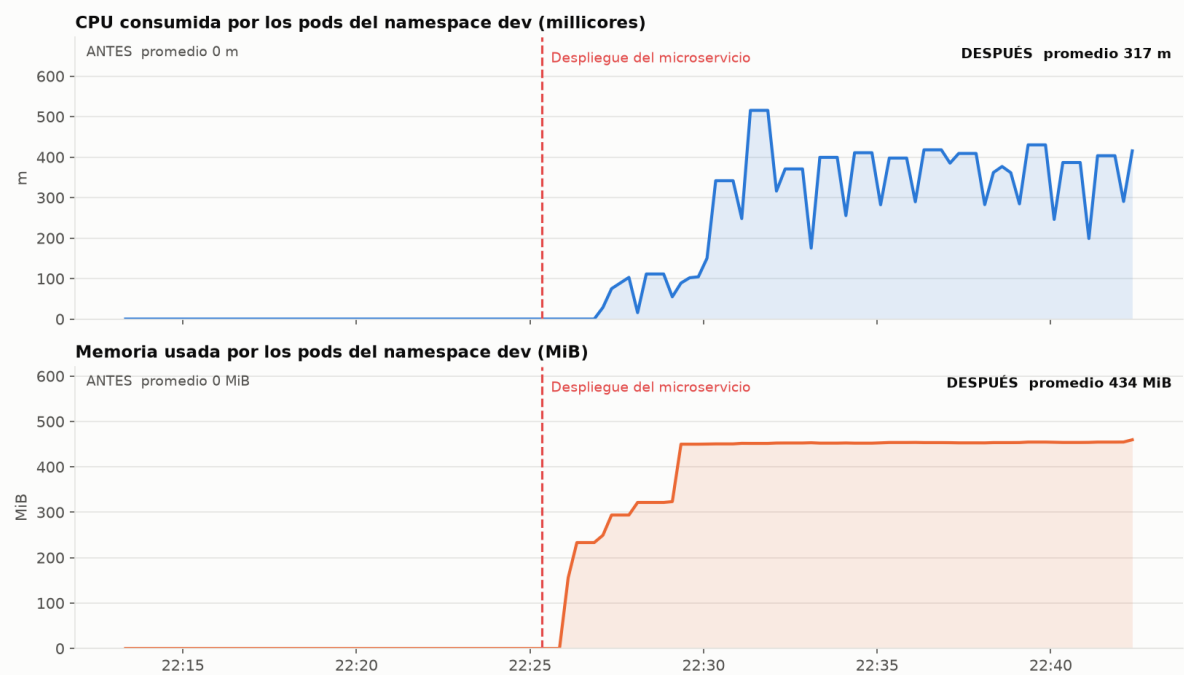
Nodo aks-acuna-dev-eastus-01: antes y después de desplegar el microservicio



Fuente: Prometheus (node-exporter), consulta query_range paso 15s. Línea roja = kubectl apply del Deployment podinfo.

Figura 6. CPU y memoria del nodo obtenidas de Prometheus; la línea roja marca el kubectl apply del microservicio.

Microservicio podinfo (+ generador de tráfico): consumo propio de CPU y memoria



Fuente: Prometheus (cAdvisor del kubelet). Antes del despliegue el namespace dev no existía (consumo 0).

Figura 7. Consumo propio de los pods del namespace dev (microservicio + generador de tráfico) medido por cAdvisor.

evidencias/15-top-despues.txt (kubectl top tras el despliegue; comparar con 06-top-antes.txt)

```
# Estado DESPUÉS de desplegar el microservicio (2026-09-23 22:42:07 -05)
$ kubectl top nodes
NAME                                CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
aks-default-12912847-vmss000000    796m        41%      2441Mi          87%

$ kubectl top pods -n dev
NAME                                CPU(cores)   MEMORY(bytes)
load-generator-7d9dc6dcc8-24tt7    72m          0Mi
load-generator-7d9dc6dcc8-4k2ng    84m          1Mi
load-generator-7d9dc6dcc8-9h2dp    77m          0Mi
load-generator-7d9dc6dcc8-zfhfh    110m         0Mi
podinfo-65f46f89-hwswc             22m          148Mi
podinfo-65f46f89-lkvqr             23m          147Mi
podinfo-65f46f89-mnbsq             25m          156Mi

$ kubectl top pods -A --sort-by=memory | head -12
NAMESPACE   NAME                                CPU(cores)   MEMORY(bytes)
monitoring   grafana-6d9b59c579-vgg9q           5m           171Mi
monitoring   prometheus-server-fcf964c77-8jl7w   3m           164Mi
dev          podinfo-65f46f89-mnbsq             25m          156Mi
dev          podinfo-65f46f89-hwswc             22m          148Mi
dev          podinfo-65f46f89-lkvqr             23m          147Mi
monitoring   loki-0                              4m           140Mi
kube-system  metrics-server-7c4d5d8fdb-7kv9b     2m           107Mi
monitoring   loki-promtail-2lx5l                 23m          104Mi
kube-system  csi-azurefile-node-nrp6t            2m           83Mi
kube-system  kube-proxy-ntt85                    1m           68Mi
kube-system  csi-azuredisk-node-r7rdn            1m           65Mi

$ kubectl get pods -n dev
NAME                                READY   STATUS    RESTARTS   AGE
load-generator-7d9dc6dcc8-24tt7    1/1     Running   0           14m
load-generator-7d9dc6dcc8-4k2ng    1/1     Running   0           14m
load-generator-7d9dc6dcc8-9h2dp    1/1     Running   0           14m
load-generator-7d9dc6dcc8-zfhfh    1/1     Running   0           14m
podinfo-65f46f89-hwswc             1/1     Running   0           16m
podinfo-65f46f89-lkvqr             1/1     Running   0           16m
podinfo-65f46f89-mnbsq             1/1     Running   0           16m
```

8. Conclusiones

- Se aprovisionó un clúster AKS con Terraform siguiendo la nomenclatura del Cloud Adoption Framework de Azure, y se comprobó con terraform plan que el código y la infraestructura real coinciden.
- Prometheus, Loki y Grafana se instalaron con Helm en el namespace monitoring. La configuración de Grafana (data sources y dashboards) se dejó como código, por lo que el entorno arranca listo para usarse y es reproducible.
- El microservicio podinfo se desplegó con Deployment + Service, quedó en Running con IP pública y su tráfico se reflejó de inmediato tanto en los logs (Loki) como en las métricas (Prometheus) dentro de Grafana.
- El impacto del microservicio es medible: la CPU del nodo pasó de 12,8 % a 37,8 % y la memoria de 37,1 % a 51,6 %, mientras que los pods del namespace dev consumen 317 millicores y 434 MiB.
- El stack Open Source cubrió todos los requisitos sin costo de licencia; el costo real es el tiempo de operación, que en producción debe compararse con una alternativa gestionada o de pago.

9. Anexos

9.1 Generador de tráfico

k8s/load-generator.yaml

```
# Generador de trafico hacia el microservicio (simula usuarios) para
# evidenciar el aumento de CPU/memoria y la generacion de logs 200 y 404.
apiVersion: apps/v1
kind: Deployment
metadata:
  name: load-generator
  namespace: dev
  labels:
    app: load-generator
spec:
  replicas: 4
  selector:
    matchLabels:
      app: load-generator
  template:
    metadata:
      labels:
        app: load-generator
    spec:
      containers:
        - name: load
          image: busybox:1.36
          command: ["/bin/sh", "-c"]
          args:
            - |
              while true; do
                wget -q -T 2 -O /dev/null http://podinfo.dev.svc.cluster.local/ || true
                wget -q -T 2 -O /dev/null http://podinfo.dev.svc.cluster.local/api/info || true
                wget -q -T 2 -O /dev/null http://podinfo.dev.svc.cluster.local/status/404 || true
                sleep 0.01
              done
          resources:
            requests:
              cpu: 20m
              memory: 16Mi
            limits:
              memory: 32Mi
```

9.2 Consultas PromQL de los dashboards

```
# CPU del nodo (%)
100 - (avg by (instance) (irate(node_cpu_seconds_total{mode="idle"}[5m])) * 100)
# Memoria del nodo (%)
(1 - (node_memory_MemAvailable_bytes / node_memory_MemTotal_bytes)) * 100
# Pods en Running
sum(kube_pod_status_phase{phase="Running"})
# Requests/s del microservicio por código HTTP
sum by (status) (rate(http_requests_total{namespace="dev", app="podinfo"}[2m]))
# CPU y memoria de los pods del namespace dev
sum(rate(container_cpu_usage_seconds_total{namespace="dev", container!=""}[2m]))
sum(container_memory_working_set_bytes{namespace="dev", container!=""})
```

9.3 Limpieza del entorno

```
kubectl delete -f k8s/load-generator.yaml -f k8s/microservicio.yaml
helm uninstall grafana loki prometheus -n monitoring
terraform destroy
```

Al finalizar las evidencias el clúster debe destruirse con terraform destroy para no seguir consumiendo créditos de Azure (nodo, discos y las IP públicas de los LoadBalancers).